

ITI5120 Industry Experience Studio Project, Part A
Data Management Plan

EV-FLOW

Electric Vehicle Forecasting and Location Optimization Wayfinder

GROUP NAME
4 JKT

Avirzan Rivai (34400729)
Beta Ningtiyas (34337547)
Muhammad Aidil Akbar (35775475)
Muhammad Arief Kurniawan (35676884)
Nicholas (34392815)

Contents

1. Datasets overview

To address the growing demand for accessible electric vehicle (EV) infrastructure across Java, Indonesia, with the Iteration 1 build scoped to Jabodetabek, the open datasets listed below will be used to help EV drivers locate Public EV Charging Stations (SPKLU), plan optimal routes to the nearest available station, and gain insight into EV adoption trends. The datasets combine official government records, crowdsourced charging-station data, geographic information, and real-time routing services.

Data sources

Data source name	Physical access	Source update freq.	System update freq.	Granularity	Copyright and licensing details
Jakarta SPKLU (Public EV Charging Stations)	CSV	Periodically (on government update)	Monthly	High	Jakarta Open Data. Attribution is required, crediting Jakarta Smart City or Dinas Perhubungan. Data is provided for public use with no warranty.
Open Charge Map (OCM)	API (JSON)	Continuous (crowdsourced)	Real-time or hourly cache	High	CC BY-SA 4.0. Attribution requires appropriate credit and a link to the licence. ShareAlike requires derived data to be distributed under the same licence.
OpenStreetMap (amenity=charging_station)	API (Overpass)	Continuous (community edits)	Daily	High	Open Database License (ODbL 1.0), © OpenStreetMap contributors. Attribution and ShareAlike are required for any derived database.
Jakarta BPS EV and transport statistics	CSV	Annually	Annually	Medium	BPS open data. Public use is permitted with attribution to Badan Pusat Statistik Provinsi DKI Jakarta.
Indonesia Electric Vehicle (EV) Dataset 2026 (Kaggle)	CSV	Static (one-off 2026 snapshot)	Never (one-time import)	Medium	Per Kaggle dataset terms, open for non-commercial research. Attribution to the dataset author is required when published or redistributed.
Geofabrik OSM Extract for Java (Indonesia)	PBF or OSM file download	Daily	Weekly	High	Open Database License (ODbL 1.0), © OpenStreetMap contributors.

Data source name	Physical access	Source update freq.	System update freq.	Granularity	Copyright and licensing details
					Geofabrik provides the extract free of charge under the same licence as the source data.
OSRM Routing API (Project OSRM v5.24)	API (HTTP or JSON)	Real-time	Real-time per request	High	BSD-2-Clause licence for the software. The public demo server has a fair-use policy. A self-hosted instance is recommended for production use.
Xendit payment gateway (sandbox or test mode)	API (test)	Not applicable (test environment)	Per simulated transaction	Per transaction	Commercial API used only in sandbox or test mode for the payment simulation, with no real funds and no production credentials. Subject to Xendit terms of service.

1.1 Open Data Sources

Jakarta SPKLU Public EV Charging Stations (CSV): https://satudata.jakarta.go.id/open-data/detail?kategori=dataset&page_url=data-stasiun-pengisian-kendaraan-listrik-umum-spkl&data_no=1

Open Charge Map (API): <https://github.com/openchargemap/ocm-system>

OpenStreetMap Charging Station Tag (API):
https://wiki.openstreetmap.org/wiki/Tag%3Aamenity%3Dcharging_station

Jakarta BPS EV and Transport Statistics (CSV): <https://jakarta.bps.go.id/en/statistics-table/3/V1ZSbFRUY3ITbFpEYTNsVWNGcDZjek53YkhsNFFUMDkiMw>

Indonesia Electric Vehicle (EV) Dataset 2026 on Kaggle (CSV):
<https://www.kaggle.com/datasets/arsan5623/indonesia-electric-vehicle-ev-dataset-2026>

Geofabrik OSM Extract for Java (Indonesia):
<https://download.geofabrik.de/asia/indonesia/java.html>

OSRM Routing API: <https://project-osrm.org/docs/v5.24.0/api/>

Xendit (payment gateway, sandbox or test mode): <https://www.xendit.co/>

1.2 Datasets by epic and user role

EV-FLOW serves two user types. B2C users are EV drivers. B2B users are business and infrastructure planners. Iteration 1 delivers value to drivers first, through Epic 1.0 (charger discovery and filtering) and Epic 6.0 (tariff and payment simulation). The planner-facing epics, together with the driver epics that depend on routing or live station data, are deferred to later iterations. The table below colours each data source by the epic it serves and the iteration in which it becomes active.

Epic and feature	Iteration	Active or deferred data sources
B2C, EV Drivers (Iteration 1 focus)		
Epic 1.0 Discovery and Filtering	Iteration 1 (active)	Jakarta SPKLU, Open Charge Map, OpenStreetMap
Epic 6.0 Tariff and Payment	Iteration 1 (active)	Indonesia EV dataset, Operator tariff inputs, Xendit (sandbox)
Epic 2.0 Route and Battery	Iteration 2 (deferred)	Geofabrik OSM extract, OSRM Routing API
Epic 3.0 Queue and Readiness	Later (deferred)	Live station telemetry or queue feed (gap in open data)
B2B, Planners (deferred to Iteration 3)		
Epic 4.0 Demand Heatmap	Iteration 3 (deferred)	Jakarta BPS, population, mobility, POI
Epic 5.0 Site Suitability and ROI	Iteration 3 (deferred)	BPS plus spatial, competitor, grid capacity

Colour indicates the epic. The two banded rows separate the user roles, B2C drivers and B2B planners. Active Iteration 1 epics are shown in full colour and deferred epics are tinted.

2. Iteration 1

2.1 Data usage

Iteration 1 establishes the Minimum Viable Product (MVP) using only external open data. It supports the initial map experience and the payment-simulation features by isolating the attributes needed for mapping, filtering, and session and cost calculation. The data-preparation and storage sections that follow describe the complete plan across all three iterations, and each dataset is labelled with the iteration in which it becomes active. The entity-relationship diagram in section 2.4 shows the Iteration 1 schema only, so the routing and analytics tables that belong to later iterations are not drawn there.

For the spatial infrastructure, the Jakarta SPKLU dataset from Satu Data Jakarta provides the baseline government-verified coordinate matrix, including physical addresses, the managing operator, and initial coordinates. The OpenStreetMap charging-station tag API complements it with community-contributed real-time updates, which mitigate outdated government cycles by catching newly deployed stations within the default 5 km radius. The Open Charge Map API adds technical metadata to each map marker, including connector counts, hardware operational state, power output, and plug formats such as AC Type 2 and DC CCS Type 2, and this feeds the dynamic connector-compatibility filters.

For the calculators and the transactional workflow, the Kaggle Indonesia EV dataset populates the EVModel catalogue with the usable battery capacities, maximum intake limits, and connector types of models sold in the Indonesian market. When a user selects their model and a target State of Charge, the charging-cost calculator reads these specifications to compute the required kilowatt-hours. Public operator pricing, namely the energy rate per kilowatt-hour and a fixed service fee, is held in the Tariff table and cross-referenced against the connector power pulled from the infrastructure APIs. The simulator then validates the balance held in the user Wallet and writes an itemised Transaction record to the charging-transaction history once a session completes. The payment itself is simulated through the Xendit sandbox in test mode with no real funds, and it drives the wallet balance check, the QR-code activation, and the receipt generation.

To enforce the MVP scope, the Geofabrik OpenStreetMap extract and the OSRM Routing API are deferred to Iteration 2, where they support the RouteRequest feature and the battery-depletion routing simulator. The Jakarta BPS transport statistics are deferred to

Iteration 3, where they feed the EVStatistics table behind the demand heatmaps and the investment-suitability matrices used by automotive vendors and station owners.

2.1.1 Iteration 1 data in detail

This subsection lists the exact attributes used in Iteration 1, the source each one comes from, and the acceptance criteria (AC) and feature it supports. Only the two active epics are covered here, Epic 1.0 for charger discovery and filtering and Epic 6.0 for tariff and payment simulation.

Jakarta SPKLU (Epic 1.0, User Story 1.1)

The fields used are station name, managing operator, the full administrative address (kelurahan, kecamatan, kota or kabupaten), latitude and longitude, and operational status. These fields drive AC 1.1.1, which renders every verified operational SPKLU marker inside the default 5 km radius when the map opens at the user position.

Open Charge Map (Epic 1.0 and Epic 6.0, User Stories 1.2 and 6.1)

The fields used are connector type (for example CCS Type 2, CHAdeMO, AC Type 2), power output in kilowatts (which sets the speed tier such as ultra-fast above 150 kW), operational status, the operator, and coordinates. Connector type and power drive AC 1.2.1, the dynamic compatibility filter, and the speed value feeds the duration estimate used by the charging-cost calculator.

OpenStreetMap charging-station tags (Epic 1.0)

Community nodes add coverage where government and Open Charge Map records are thin. The socket tags (socket:type2, socket:chademo), capacity, fee, opening_hours, and access fields are normalised into the same schema as the other two sources, so the map and the filters treat every station the same way.

Indonesia EV dataset, Kaggle (Epic 6.0, User Story 6.1)

The fields used are make, model, usable battery capacity in kilowatt-hours, maximum intake power, and connector type. They populate the EVModel catalogue. When a driver picks a model and a charge target, AC 6.1.2 uses the battery capacity to compute the kilowatt-hours required to move from one State of Charge to another.

Operator tariff inputs (Epic 6.0, User Stories 6.1 and 6.4)

Compiled public pricing provides an energy rate per kilowatt-hour and a fixed service fee for each operator and power tier. These values feed the tariff comparison in AC 6.1.1, the cost estimate in AC 6.1.2, and the itemised receipt in AC 6.4.1.

Xendit sandbox (Epic 6.0, User Stories 6.2 to 6.4)

The payment runs entirely in test mode with no real funds. It backs the wallet balance check in AC 6.2.1, the transaction ledger entry in AC 6.2.2, the QR session validation in AC 6.3.1, and the digital receipt in AC 6.4.1.

User and device inputs (Epic 1.0 and Epic 6.0)

The map uses the device GPS position. The position is rounded on the client before use and is not stored as a raw itinerary, in line with AC 1.1.2 and the location privacy mandate of UU PDP No. 27 of 2022. The driver also supplies the selected vehicle model and the charge target that the calculator needs.

2.2 Data Preparation

2.2.1 Data Cleaning and Wrangling

Jakarta SPKLU (CSV)

- **Cleaning:** removed duplicate rows that arose when the same SPKLU site is published under multiple administrative codes, standardised text fields (operator name, address, kecamatan) to Title Case, trimmed whitespace, and corrected obvious typographical errors in postcodes.

- **Wrangling:** split the combined connector field into separate columns for connector type, power rating (kW) and quantity. Rows missing latitude or longitude were geocoded using the address and the OSM Nominatim service, within its bulk-use policy of roughly one request per second and with attribution. Each station was tagged with its parent kabupaten or kota to support aggregation.

Open Charge Map (API)

- **Data loading:** data is fetched in pages of 500 records from the OCM API, filtered by country code (ID) and a bounding box covering Jabodetabek, which is the Iteration 1 service area.
- **Deduplication against SPKLU:** a station from OCM is treated as a duplicate of an SPKLU row when the two points lie within 50 m of each other and share at least one connector type. Duplicates are merged, keeping the more recent OCM verification date.
- **Field mapping:** OCM attributes are mapped to the internal schema. Title becomes station_name, AddressInfo latitude and longitude become lat and lon, ConnectionType title becomes connector_type, PowerKW becomes power_kw, and StatusType IsOperational becomes is_operational.
- **Caching:** API responses are written to a local cache with a 1-hour TTL to reduce request volume and respect OCM rate limits.

Xendit payment sandbox (Epic 6.0)

- **Test-mode integration:** the simulated wallet, the QR activation, and the receipts call the Xendit sandbox in test mode. No real funds move and no production payment credentials are stored.
- **Records written:** each simulated checkout writes a Transaction with energy cost, service fee, total, payment method, and timestamp, and it updates the Wallet balance after a balance-validation check.

OpenStreetMap (amenity=charging_station)

- **Extraction:** an Overpass query pulls all nodes and ways tagged amenity=charging_station within the Jabodetabek bounding box, and the result is exported as GeoJSON.
- **Cleaning:** nodes with no operator and no recent edit (older than 2 years) are flagged as low-confidence and excluded from the live layer.
- **Wrangling:** OSM tags (socket:type2, socket:chademo, capacity, fee, opening_hours) are normalised into the same schema used for SPKLU and OCM.

Geofabrik OSM Extract for Java (Iteration 2)

- **Loading:** the daily java-latest.osm.pbf extract is downloaded and imported into a local PostgreSQL or PostGIS database using osm2pgsql.
- **Filtering:** only highway, place and amenity tags relevant to routing and POI display are retained, dropping the rest to keep the database size manageable.
- **Preparation for OSRM:** a separate copy of the PBF is processed with osrm-extract, osrm-partition, osrm-customize and osrm-contract so the routing engine can serve requests against the same data version used in the database.

Jakarta BPS and Kaggle Indonesia EV 2026 (Iteration 3 for BPS analytics)

- Both datasets are loaded with pandas. Column names are standardised to snake_case, numeric fields are coerced to numeric types, and year columns are parsed to integers.
- Filtering keeps only rows relating to electric vehicles (sepeda motor listrik, mobil listrik) from the BPS file. The Kaggle dataset is filtered to the columns relevant to this iteration (province, year, vehicle_type, count, manufacturer).

- Integration aggregates the two datasets to one row per province-year before merging on province and year, producing a single time-series view of EV uptake in Java, which is then exported to CSV (ev_uptake_java.csv) ready for import into MySQL. Note that the BPS table cited covers DKI Jakarta only. Province-level coverage for the rest of Java requires the corresponding BPS tables for those provinces.

2.3 Data storage

The storage strategy uses three mechanisms, a pattern known as polyglot persistence, with each mechanism chosen for a class of data and activated across the three iterations.

Charging-infrastructure and spatial data is stored in PostgreSQL with the PostGIS extension. This covers the consolidated ChargingStation table built from the SPKLU, OCM and OSM sources during cleaning, together with Location, Connector, Operator, Tariff and AvailabilitySnapshot. PostGIS is purpose-built for the spatial queries at the centre of the application, namely nearest-neighbour, within-radius, and point-in-polygon, so the core query for chargers within 5 km or within a 30-minute drive runs here against spatial indexes such as ST_DWithin. AvailabilitySnapshot accumulates periodic station-status snapshots from Iteration 1 onward, so the time-of-day foresight in Iteration 2 has historical data to learn from. In Iteration 2, the Geofabrik road-network extract and the OSRM routing tile-set, built from the same data version, are added to this database to support routing.

User-facing transactional data and analytics are stored in MySQL. This covers User, Wallet, Vehicle, EVModel, ChargingSession and Transaction in Iteration 1, and the denormalised EVStatistics or merged EV-uptake table in Iteration 3. MySQL provides relational integrity for the account, session and transaction tables and suits the aggregate queries of the analytics dashboard. Because the charging-infrastructure tables sit in PostGIS while the session and transaction tables sit in MySQL, references that cross the boundary, for example ChargingSession.StationID and Transaction.TariffID, are resolved at the application layer rather than by database-enforced foreign keys. If a single enforced schema is preferred, the operational tables can instead be consolidated onto PostgreSQL with PostGIS, which supports both relational and spatial data, leaving MySQL purely as the analytics store.

Real-time data obtained through APIs is handled by a Redis caching layer. Live Open Charge Map station status in Iteration 1 and OSRM routing responses in Iteration 2 are cached with explicit TTLs, one hour for OCM and five minutes for OSRM routes, to minimise latency, reduce upstream traffic, and keep the application within the fair-use limits of the public OSRM demo server. The persisted AvailabilitySnapshot rows are written from this cached live status on a schedule, so the cache stays temporary while the history is retained in PostGIS.

2.4 Database design

The entity-relationship (ER) diagram below shows the Iteration 1 entities and how they relate to each other. Each entity is coloured by the epic it serves, Epic 1.0 for charger discovery and filtering and Epic 6.0 for tariff and payment, so the diagram also works as a map from data to feature. It can be used to better understand the structure of the database.

[Insert the epic-coloured Iteration 1 ER diagram here]

Figure 1. EV-FLOW Iteration 1 entity-relationship diagram, coloured by epic.

Key relationships

- A User owns zero or many Vehicles. Each Vehicle is owned by exactly one User and references exactly one EVModel, the catalogue of technical specifications sourced from the Kaggle dataset.
- A User has exactly one Wallet and may initiate many ChargingSessions and make many Transactions.

- A ChargingStation is located at exactly one Location and is managed by one Operator. An Operator manages many Stations and sets many Tariffs.
- A ChargingStation has one or many Connectors, and each Connector belongs to exactly one Station.
- A ChargingSession links one User, one Station and one Connector to a single recorded charging event, and it is billed as exactly one Transaction. Each Transaction applies the rate from one Tariff.

Design notes

- User credentials are stored as a salted PasswordHash rather than a plaintext password.
- Vehicle specifications are held once in the EVModel catalogue and referenced by Vehicle, so the cost calculator reads the specifications from a single source rather than duplicating them per user.
- Tariff, Transaction and Wallet give the payment-simulation feature a place to store operator pricing, an itemised transaction history, and simulated wallet balances.
- Transaction carries a PaymentMethod field and an optional gateway reference, so a receipt can record how a simulated session was paid, using the Xendit sandbox in Iteration 1.

2.5 Data Analytics

2.5.1 Hindsight Insights

- **EV uptake and charging-station growth:** the merged BPS and Kaggle datasets provide a historical view of how EV registrations have grown across the provinces of Java. By comparing these counts year on year against the number of public charging stations recorded in SPKLU and OCM, users and policymakers can see how charging infrastructure has kept pace with the vehicle fleet.
- **Coverage gaps:** aggregating stations by kecamatan and overlaying the result with population data, sourced from BPS, reveals areas that have historically been under-served by public charging. This informs both user expectations, such as planning ahead in a given area, and any future site-selection work.

2.5.2 Foresight Insights

- **Charging-station availability:** live Open Charge Map status data combined with the time-of-day patterns observed in previous requests lets the application warn users when a station is likely to be busy or temporarily out of service, before they make the trip.
- **Trip planning:** real-time routing from OSRM allows users to anticipate travel time and distance to a candidate station, taking the current road network into account. This is particularly useful for longer inter-city journeys across Java where range anxiety is a major concern.

2.5.3 Insights

- **Location-based recommendations:** combining the current GPS position with the consolidated station table and OSRM routes lets the application return a ranked list of nearby chargers that match the user connector type, for example only CCS2 fast chargers within a 30-minute drive.
- **Charging-time estimation:** using the connector PowerKW from the station record and the battery capacity from the EVModel catalogue, the application can estimate how long a charge will take and display it alongside the route information.
- **Charging-session tracking:** storing each ChargingSession allows users to review their charging history, compare average session length and energy delivered across

stations, and identify the chargers that work most reliably for their vehicle. Aggregated and anonymised, this same data feeds back into the foresight insights described above.